

Firewall y Filtrado de Capa 7 (nftables + Squid)

Marco teorico: De iptables a nftables

Por qué iptables está obsoleto?

Por dos décadas Iptables fue el estándar para la gestión de paquetes en Linux, pero esa arquitectura presenta limitaciones estructurales que obligaron al reemplazo en los actuales sistemas operativos.

1. **Duplicación de código e inconsistencia de herramientas:** iptables necesitaba binarios y módulos independientes para cada protocolo: iptables (IPv4), ip6tables (IPv6), arptables (ARP) y ebtables (Ethernet Bridging). Esto hacía difícil la administración y obligaba a mantener reglas duplicadas para IPv4 e IPv6.
2. **Degradación de rendimiento:** iptables evalúa las reglas en una cadena de manera lineal, una por una. Cuando en una infraestructura manejas listas extensas de bloqueo (miles de direcciones IP o subredes), el procesador agarra y compara cada paquete contra cada entrada de la lista, generando latencia y un consumo altísimo de CPU.
3. **Falta de operaciones eficientes:** Para modificar o agregar una regla en iptables tenes que copiar toda la tabla de reglas desde el espacio de kernel al espacio de usuario, modificarla y reescribirla por completo en el kernel. En entornos con reconfiguraciones dinámicas frecuentes, esto tendría latencia de red y problemas de desincronización.
4. **Estructura fija y rígida:** Las tablas (filter, nat, mangle) y las cadenas que vienen por default (INPUT, OUTPUT, FORWARD) están de forma fija en el núcleo, habiendo o no reglas adentro de ellas, esto consume recursos de memoria de forma innecesaria.

1. La solución actual: Nftables

Nftables es el subsistema del kernel linux (del proyecto Netfilter) que reemplaza a iptables. Algunas características son:

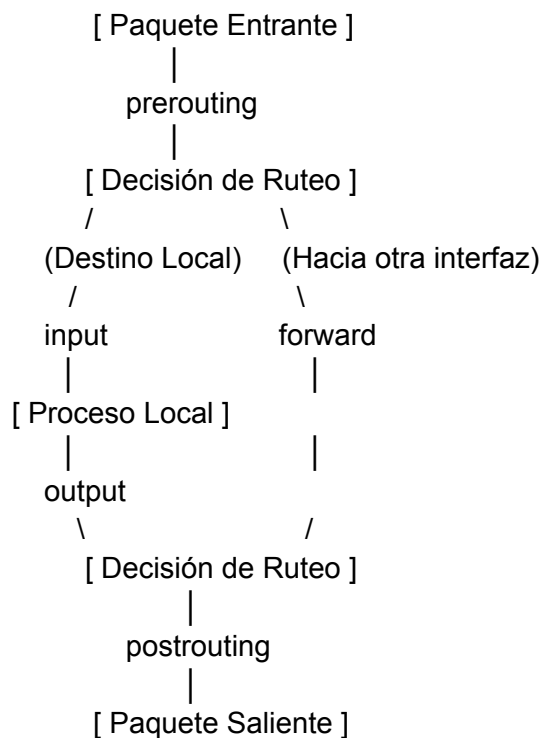
1. **Familias de protocolos juntas:** Podes manejar IPv4 e IPv6 en una misma tabla por la familia unificada inet. Una sola regla sirve para ambas pilas de red.
2. **Máquina virtual de bytecode en el kernel:** Las reglas que se escriben se compilan en un conjunto de instrucciones de bytecode muy livianas que una VM interna del kernel ejecuta directamente, optimizando el uso de la CPU.
3. **Estructuras de datos avanzadas (Sets, Maps y Dictionaries):** Nos deja agrupar miles de direcciones IP, puertos o subredes en conjuntos. El kernel hace búsquedas con complejidad $O(1)$ o $O(\log N)$, entonces mantiene el mismo rendimiento de procesamiento y no importa si se evalúa 1 IP o 100.000 IPs.
4. **Tablas y cadenas configurables a medida:** En nftables no existen tablas ni cadenas creadas por defecto. El sysadmin pone únicamente las tablas necesarias y le asigna los enganches (hooks) del kernel y los índices de prioridad exactos.
5. **Capa compatibilidad (iptables-nft):** En distros actuales (como Debian 11 a 13, o RHEL 8 a 10), los comandos en iptables usan internamente la capa de traducción iptables-nft, que convierten las instrucciones al motor de nftables.

2. Arquitectura de nftables: Enganches (Hooks) y Prioridades

Para manejar el tráfico, nftables conecta sus cadenas a los enganches (hooks) del subsistema de red del kernel:

1. prerouting: agarra el paquete cuando entra por la interfaz de red, antes de la decisión de enrutamiento. Utilizado para Destination NAT (DNAT) o redirecciones.
2. input: Procesa los paquetes donde el destino final es el host local.
3. forward: procesa paquetes que atraviesan el host (cuando hace de router o puerta de enlace) hacia otra red.
4. output: Procesa paquetes generados localmente por el propio s.o hacia la red.
5. postrouting: Captura el paquete antes de salir por la interfaz física, después la decisión de enrutamiento. se usa para Source NAT (SNAT) y enmascaramiento (Masquerade).

Flujo en Netfilter (nftables):



3. Estado de Conexión (Conntrack) y Políticas de Seguridad

Stateful Inspection: Mediante el módulo de seguimiento de conexiones (conntrack), el firewall agarra y clasifica cada paquete según el estado de la sesión:

1. new: Solicitud para iniciar una nueva conexión (por ej, TCP SYN).
2. established: Tráfico perteneciente a una sesión bidireccional ya negociada.
3. related: Conexiones secundarias asociadas a una sesión activa (errores ICMP, canales de datos).
4. invalid: Paquetes que no son de ninguna sesión registrada o tienen algo raro (anomalías) en sus cabeceras. Se descartan rapido(drop).

Estrategia Whitelist (Default DROP): La política de seguridad cuando estamos en producción te exige configurar las cadenas con una política default de rechazo o descarte (policy drop). Todo el tráfico que entra o esta en tránsito se deniega por defecto, habilitando únicamente los puertos, protocolos e IPs requeridos mediante reglas explícitas.

4. Traducción de direcciones (NAT): SNAT vs MASQUERADE

Cuando un firewall Linux da acceso a Internet a una red localhost, cambia la IP de origen en la cadena que esta asociada al hook postrouting:

SNAT (Source NAT estático): Se especifica la IP pública fija de salida. Es lo más eficiente en servidores de producción con IP estática, porque el kernel no tiene que consultar el estado de la interfaz en cada paquete.

masquerade (Enmascaramiento dinámico): Se usa cuando la interfaz WAN tiene una dirección IP dinámica (por ej, DHCP de un proveedor ISP). El kernel evalúa la IP activa que pasa por la interfaz en cada paquete, esto sube un poco el procesamiento respecto a SNAT.

5. El riesgo de la desprotección en IPv6

Un fallo común cuando migras o configuras firewall es aplicar reglas solo en IPv4. Los S.O actuales usan Dual-Stack y generalmente dan prioridad a las resoluciones AAAA (IPv6). Si el firewall no filtra la IPv6, el tráfico puede omitir las restricciones de red y hacer conexiones directas sin control. Al usar nftables con inet, una única regla aplica las mismas restricciones sobre IPv4 e IPv6 en paralelo.

Parte 2: El Proxy de Capa 7: Squid en Entornos Actuales

1. Caché Web y la Función Actual de Squid

Antes, los servidores proxy como Squid se instalaban para guardar en caché páginas web (HTTP) y ahorrar ancho de banda. Hoy en día, eso ya no sirve porque más del 95% del tráfico web va cifrado por HTTPS (TLS/SSL).

Como la data viaja cifrada de punta a punta, el motor de caché no puede leer ni guardar esos objetos. Por eso, el rol real de Squid en producción hoy es:

1. **Filtrado de Salida (Egress Filtering):** Limitar a qué sitios o servicios de Internet pueden salir las máquinas de la red interna o los servidores de backend.
2. **Control de Acceso con Políticas (ACLs):** Restringir la navegación por horarios, subredes, dominios o directamente por usuario autenticado.
3. **Trazabilidad y Auditoría (SOC / Forense):** Dejar un registro exacto de las peticiones salientes en `/var/log/squid/access.log` para detectar tráfico raro, malware o si un equipo infectado está tratando de hablar con un servidor C2 (Comando y Control).

2. **Estrategias para filtrar tráfico HTTPS** Para analizar el tráfico cifrado, tenés dos caminos posibles:

A. SSL Bumping (Inspección MitM Completa)

Cómo funciona: Squid hace literalmente un ataque de Man-in-the-Middle. Atrapa la conexión TLS del cliente, arma su propia sesión con el servidor remoto, lee el contenido descriptado y le genera en tiempo real un certificado falso al usuario, firmado por la CA del propio proxy.

Los problemas de esto:

1. **Despliegue de certificados:** Tenés que instalar a mano (o por GPO/scripts) la clave pública de la CA raíz de Squid en todas las máquinas de la red. Si no lo hacés, los navegadores te dan la clásica pantalla bloqueando la web.
2. **Certificate Pinning y apps rotas:** Muchas apps de hoy (bancos, WhatsApp, llamadas VoIP) traen sus certificados fijos en el código (Certificate Pinning). Apenas notan que el proxy alteró el certificado, cortan la conexión al instante.
3. **Consumo de recursos:** Descriptar, leer y volver a encriptar todo el tráfico exige una bestialidad de CPU y memoria en el servidor.

B. SNI Peeking (Server Name Indication) (Estándar Actual)

1. **Cómo funciona:** En el primer saludo de la conexión (TLS Handshake), el cliente manda un paquete en texto plano llamado Client Hello. Ese paquete trae el campo SNI (Server Name Indication), que básicamente dice a qué dominio quiere ir (ej: youtube.com).
2. **El truco:** Squid solo "revisa" ese primer paquete sin tocar ni desenscriptar el resto del túnel cifrado. Saca el dominio del SNI, se fija en tus Listas de Control de Acceso (ACLs) y ahí decide si aprueba la conexión o si la voltea mandando un TCP Reset.
3. **Ventajas:** Te permite bloquear sitios HTTPS por dominio sin matar el rendimiento del server, sin romper el cifrado y sin la pesadilla de andar instalando certificados en las PCs.

3. Modos de Despliegue del Proxy

A. **Proxy Explícito:** Le decís al navegador o al sistema operativo (manualmente o con un archivo WPAD) que mande todo directo a la IP y puerto de Squid (el 3128 por default).

Ventaja: Es el esquema más limpio. Te deja pedir usuario y contraseña (usando LDAP, Active Directory o Kerberos), así en los logs ves exactamente quién fue el que entró, no solo una IP.

B. **Proxy Transparente / Interceptado:** nftables agarra el tráfico web y lo redirige a la fuerza al puerto de Squid, sin que el usuario tenga que configurar nada.

Uso: Se usa mucho en redes de invitados (hoteles, escuelas) donde no controlás las computadoras. El problema es que falla bastante con HTTPS y te quita la posibilidad de pedirle usuario y contraseña a la gente.

4. **Lógica de Evaluación de ACLs en Squid** Igual que el firewall de red, Squid lee las listas de control de arriba para abajo (Top-Down). Analiza las reglas y ejecuta la acción en la primera coincidencia que encuentra:}

1. **Definición de las ACLs (Mapeo de objetos):**

```
acl red_interna src 10.0.10.0/24
```

```
acl dominios_prohibidos dstdomain .casino.com .malware-db.net
```

```
acl horario_laboral time MTWHF 08:00-17:00
```

2. **Aplicación de Reglas (http_access):**

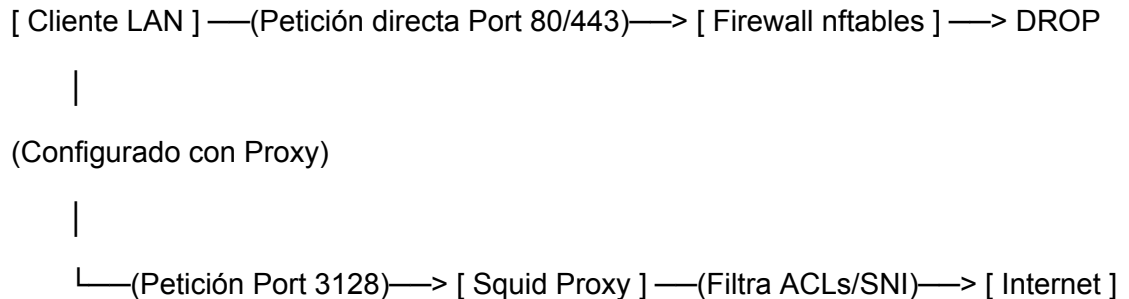
```
http_access deny dominios_prohibidos
```

```
http_access allow red_interna horario_laboral
```

```
http_access deny all
```

~ **La red de seguridad:** La regla `http_access deny all` tiene que ir sí o sí al final del archivo para asegurarte de que, si un paquete no coincidió con nada arriba, se bloquee por defecto.

5. Integración de Seguridad (nftables + Squid) Para evitar que algún usuario "vivo" le saque la configuración del proxy a su navegador y salga a internet libremente, se arma este esquema:



nftables bloquea la salida directa: En la cadena forward metés un drop a cualquier intento de las PCs de salir derecho a los puertos 80 o 443.

nftables solo deja salir a Squid: La única IP autorizada a salir a internet por el 80 y 443 es la del propio servidor Proxy.

El embudo centralizado: Los usuarios no tienen otra opción que pasar por el puerto 3128 de Squid. Así te asegurás de inspeccionar los dominios con SNI Peeking y dejar todo guardado en el log de auditoría.

Práctica: Nftables y Squid

Vamos a implementar una arquitectura Zero Trust lo más real a como se usa en producción destripando el "por qué" de cada línea.

Entonces para esta practica usaremos debian 13 server en tres distintas VM:

1. Una sera la maquina virtual donde estará el gateway/firewall con dos placas de red, una WAN con salida a internet y otra con LAN hacia la red local.
2. La segunda es donde habrá servicios como DHCP, NFS, SFTP, entre otros (en esta máquina se encuentran todos los servicios que he documentado en el repositorio). Esta maquina no se usara como tal en el laboratorio, sera simbólica, representa los servicios a proteger en una infraestructura de producción real
3. Y la tercera será un cliente LAN.

Empezaremos por la VM1 gateway/firewall.

Parte 1: convertir la VM1 en un Router

El kernel de Linux por seguridad descarta los paquetes que no van dirigidos a él. Como la VM1 va a ser el intermediario entre la WAN y LAN, tenemos que hacer que reenvie el trafico.

Para esto habilitamos el reenvío de paquetes en memoria:

```
~ echo 1 > /proc/sys/net/ipv4/ip_forward
```

También es necesario hacerlo persistente tras un reinicio o actualizacion:

```
~ echo "net.ipv4.ip_forward=1" > /etc/sysctl.d/99-router.conf
```

```
~ sysctl --system
```

Esto creará un archivo y le pone la linea ip forward adentro, nos asegura que lea al final y pise cualquier otra regla. El otro comando hará que el kernel lea el directorio.

```
root@UTN-Lucka:~# echo "net.ipv4.ip_forward=1" > /etc/sysctl.d/99-router.conf
root@UTN-Lucka:~# sysctl --system
* Applying /usr/lib/sysctl.d/10-coredump-debian.conf ...
* Applying /usr/lib/sysctl.d/50-default.conf ...
* Applying /usr/lib/sysctl.d/50-pid-max.conf ...
* Applying /etc/sysctl.d/99-router.conf ...
```

Parte 2: Instalar y configurar Squid

Squid será nuestro filtro de tráfico, será el que “vigile” la red.

Lo instalamos con:

```
~ apt update
```

```
~ apt install squid nftables curl
```

Ahora el objetivo es configurar el archivo, antes creamos un backup:

```
~ mv /etc/squid/squid.conf /etc/squid/squid.conf.backup
```

Para que hacemos esto? El archivo que trae Squid por defecto tiene casi 8000 lineas, si tenemos un problema, tratar de leer ese archivo con la mayoría de lineas comentadas es difícil. En producción siempre se renombra el original como backup y se usa un archivo limpio y minimalista, entonces si se rompe algo, leemos solo lo que escribimos nosotros.

Configuramos el archivo.

En Squid primero se definen las ACL (access control lists) se define quienes y que existen. Despues se pone las reglas de acceso (TOP-DOWN) Squid lee de arriba hacia abajo, el primero que coincide “gana”.

```
~ vim /etc/squid/squid.conf
```

```
# —PUERTO Y LOGS
```

```
http_port 3128
```

```
access_log daemon:/var/log/squid/access.log squid
```

```
# —DEFINICIÓN DE ACLs
```

```
acl red_local src 192.168.122.0/24
```

```
# Usamos regex para interceptar mejor el SNI del HTTPS
```

```
acl dominios_prohibidos url_regex -i facebook\.com instagram\.com
```

```
acl puertos_seguros port 80 443
```

```
acl SSL_ports port 443
```

```
acl CONNECT method CONNECT
```

```
# —REGLAS DE ACCESO (TOP-DOWN)
```

```
# seguridad interna del proxy
```

```
http_access deny !puertos_seguros
```

```
http_access deny CONNECT !SSL_ports
```

```
# Bloquear la lista negra
```

```
http_access deny dominios_prohibidos
```

```
# dejar pasar a nuestra red local
```

```
http_access allow red_local
```

```
# 4. default DROP (Cerrar la puerta al resto)
```

```
http_access deny all
```

Desglose de la configuracion:

`acl red_local src 192.168.122.0/24:` le enseñamos a Squid cómo es la LAN.

`acl puertos_seguros port 80 443:` Definimos que la navegación web normal solo ocurre en estos dos puertos.

`acl dominios_prohibidos url_regex -i facebook\.com instagram\.com:` Facebook y Google ahora usan dominios dinámicos y CDN, para que la regla funcione usamos una expresión regular. Esto hace que Squid busque esa palabra en cualquier parte de la petición cifrada inicial, haciéndolo infalible.

`acl CONNECT method CONNECT:` importante, Cuando un navegador quiere ir a un sitio HTTPS, no puede pedir "dame la página". Usa el método CONNECT para decirle al proxy: Abri un túnel a ciegas hacia este destino. Squid lo reconoce y aplica las reglas sobre ese túnel.

`http_access deny !puertos_seguros:` si se intenta salir por un puerto que NO sea el 80 o el 443, se bloquea de entrada. Esto evita que alguien use el proxy para hacer SSH hacia afuera o transferir archivos raros.

`http_access deny dominios_prohibidos:` "Si van a Facebook, bloquealos".

`http_access allow red_local:` "Si pasaron los filtros de arriba y vienen de mi red local, dejalos salir a Internet".

`http_access deny all:` La red de seguridad absoluta. "Si llegó hasta acá y no calza en nada, bloquealo por defecto".

Luego de esto, guardamos el archivo y reiniciamos e habilitamos el servicio:

~ systemctl restart squid

~ systemctl enable squid

```
# -PUERTO Y LOGS
http_port 3128
access_log daemon:/var/log/squid/access.log squid

# -DEFINICIÓN DE ACLs
acl red_local src 172.20.0.0/16
# Usamos regex para interceptar mejor el SNI del HTTPS
acl dominios_prohibidos url_regex -i facebook\.com instagram\.com
acl puertos_seguros port 80 443
acl SSL_ports port 443
acl CONNECT method CONNECT

# -REGLAS DE ACCESO (TOP-DOWN)
# seguridad interna del proxy
http_access deny !puertos_seguros
http_access deny CONNECT !SSL_ports

# Bloquear la lista negra
http_access deny dominios_prohibidos

# dejar pasar a nuestra red local
http_access allow red_local

# 4. default DROP (Cerrar la puerta al resto)
http_access deny all

root@UTN-Lucka:~# mv /etc/squid/squid.conf /etc/squid/squid.conf.backup
root@UTN-Lucka:~# vim /etc/squid/squid.conf
root@UTN-Lucka:~# vim /etc/squid/
conf.d/      errorpage.css      squid.conf.backup
root@UTN-Lucka:~# vim /etc/squid/squid.conf
root@UTN-Lucka:~# systemctl restart squid
root@UTN-Lucka:~# systemctl enable squid
Synchronizing state of squid.service with SysV service script with /usr/lib/systemd/s
ysd/squid.service
Executing: /usr/lib/systemd/systemd-sysv-install enable squid
```

Parte 3: “Muro perimetral” (Nftables)

Este es el candado real, Si no armamos este “muro”, el usuario simplemente podria apagar el proxy explicito en su navegador y salir libre por internet

Si un empleado de la empresa se da cuenta de que no puede entrar a Facebook porque el proxy se lo bloquea, lo primero que va a hacer es ir a la configuración de su Chrome, destildar la cajade "Usar proxy", y darle a guardar. Listo, "apagó" el proxy de su lado.

Antes de empezar verificamos el nombre de nuestra placa WAN (la que da internet, por ej enp1s0) y la de la LAN

En mi caso asi lo configure

```
# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface - WAN
allow-hotplug enp1s0
iface enp1s0 inet static
    address 192.168.122.10
    netmask 255.255.255.0
    gateway 192.168.122.1
    dns-nameservers 8.8.8.8 1.1.1.1

#segunda - interfaz LAN
allow-hotplug enp7s0
iface enp7s0 inet static
    address 172.20.0.254          #gateway de las VM/servidores
    netmask 255.255.0.0
```

```
root@UIN-Lucka:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
1000
    link/ether 52:54:00:a6:c8:96 brd ff:ff:ff:ff:ff:ff
    altname enx525400a6c896
    inet 192.168.122.10/24 brd 192.168.122.255 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::5254:00a6:c896:0000/64 scope link proto kernel
        valid_lft forever preferred_lft forever
3: enp7s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
1000
    link/ether 52:54:00:a8:0f:d0 brd ff:ff:ff:ff:ff:ff
    altname enx525400a80fd0
    inet 172.20.0.254/16 scope global enp7s0
        valid_lft forever preferred_lft forever
```

Una vez que chequeamos la configuración de las interfaces de red, pasamos a configurar el archivo de nftables:

~ vim /etc/nftables.conf

La configuración que colocaremos será la siguiente: (reemplazando mi IPs por la de su placa LAN, y al final por el nombre de su placa WAN)

```
#!/usr/sbin/nft -f
flush ruleset
table inet firewall {
    chain incoming {
        type filter hook input priority filter; policy drop;
        iif "lo" accept
        ct state established,related accept
        tcp dport 22 accept
        ip saddr 192.168.122.0/24 tcp dport 3128 accept
    }
    chain forwarding {
        type filter hook forward priority filter; policy accept;
        ip saddr 192.168.122.0/24 tcp dport { 80, 443 } drop
    }
}
table ip nat {
    chain postrouting {
        type nat hook postrouting priority srcnat;
        oifname "enp1s0" masquerade #placa WAN
    }
}
```

Explicación de la conf.

flush ruleset: Limpia la memoria del kernel. En producción, un script de firewall siempre tiene que empezar borrando todo, porque si lo ejecutás dos veces seguidas sin borrar, las reglas se duplican y el servidor colapsa.

Chain incoming: (entrada al Servidor) Esto controla quién puede hablar con la IP de la VM1.

policy drop;: Default DROP. Nadie entra.

ct state established,related accept: El Stateful Firewall! Si el servidor inició una conexión, permite que la respuesta vuelva a entrar. Sin esto, el servidor no podría ni bajar una actualización de Debian.

tcp dport 22 accept: Para que podamos entrar por SSH a administrarlo.

ip saddr 172.20.0.0/16 tcp dport 3128 accept: "Si alguien de mi LAN intenta hablarle al puerto 3128 (Squid), dejalo entrar".

chain forwarding: (embudo) Esto controla el tráfico que *atraviesa* la VM1 yendo desde la LAN hacia Internet.

`policy accept;` (Opcional, pero para este lab lo dejamos en accept general para no complicar innecesariamente el ruteo interno).

`ip saddr 172.20.0.0/16 tcp dport { 80, 443 } drop;` La regla maestra. Si un equipo intenta saltarse el proxy e ir a Internet directamente por los puertos web, el paquete choca acá y muere. Es el famoso "embudo": o va al puerto 3128 de Squid, o no sale.

`chain postrouting` (NAT) Para que las IPs privadas de la red interna (172.20.0.x) puedan navegar, necesitan disfrazarse con la IP pública de la VM1.

`oifname "enp0s3" masquerade:` "Todo paquete que logre salir de esta máquina por la placa WAN (la que da a Internet), enmascaralo con la IP de esa placa".

Cargamos la regla en el kernel:

`~ nft -f /etc/nftables.conf`

`~ systemctl enable nftables`

```
root@UTN-Lucka:~# vim /etc/nftables.conf
root@UTN-Lucka:~# nft -f /etc/nftables.conf
root@UTN-Lucka:~# systemctl enable nftables.service
Created symlink '/etc/systemd/system/sysinit.target.'
b/systemd/system/nftables.service'.
```

```
#!/usr/sbin/nft -f

flush ruleset

table inet firewall {
    chain incoming {
        type filter hook input priority filter; policy drop;

        iif "lo" accept
        ct state established,related accept
        tcp dport 22 accept
        ip saddr 172.20.0.0/16 tcp dport 3128 accept
    }

    chain forwarding {
        type filter hook forward priority filter; policy accept;

        # bloqueamos salida web directa
        ip saddr 172.20.0.0/16 tcp dport { 80, 443 } drop
    }
}

table ip nat {
    chain postrouting {
        type nat hook postrouting priority srcnat;
        oifname "enp1s0" masquerade #placa wan que sale a internet
    }
}
```

Parte 4: Prueba y auditoría

En la VM3 cliente, la colocaremos en la misma red con una ip estatica (en mi caso 172.20.0.x) y su gateway o ruta por defecto apuntara a la VM1

```
"/etc/network/interfaces" 15L, 389B written
root@UTN-Lucka:~# systemctl restart networking
root@UTN-Lucka:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel
    link/ether 52:54:00:5e:80:7b brd ff:ff:ff:ff:ff:ff
    altname enx5254005e807b
    inet 172.20.0.5/16 brd 172.20.255.255 scope global enp1s0
        valid_lft forever preferred_lft forever
    inet6 fe80::b23c:5b65:e990:e832/64 scope link tentative
        valid_lft forever preferred_lft forever
```

después haremos un ~ systemctl restart networking

Y probaremos distintas conexiones:

Prueba 1: El usuario intenta salir sin proxy configurado

~curl -I https://google.com --connect-timeout 5

(Se tiene que quedar colgado. nftables lo agarró en la cadena forwarding y descartó el paquete porque salir por el 443 directo está prohibido).

```
root@UTN-Lucka:~# curl -I https://google.com --connect-timeout 5
curl: (28) Connection timed out after 5001 milliseconds
root@UTN-Lucka:~# curl -x http://172.20.0.254:3128 -I https://www.google.com
```

Prueba 2: El usuario configura su proxy

~ curl -x http://172.20.0.254:3128 -I https://www.google.com

(Tiene que devolver HTTP/1.1 200 OK. El proxy lo dejó pasar a internet).

```
root@UTN-Lucka:~# curl -x http://172.20.0.254:3128 -I https://www.google.com
HTTP/1.1 200 Connection established

HTTP/2 200
content-type: text/html; charset=ISO-8859-1
content-security-policy-report-only: object-src 'none';base-uri 'self';script-src 'self';
.withgoogle.com/csp/gws/other-hp
accept-ch: Sec-CH-Prefers-Color-Scheme
p3p: CP="This is not a P3P policy! See g.co/p3phelp for more info."
date: Mon, 17 Aug 2026 21:06:20 GMT
server: gws
x-xss-protection: 0
x-frame-options: SAMEORIGIN
expires: Mon, 17 Aug 2026 21:06:20 GMT
cache-control: private
set-cookie: __Secure-STRP=ANmZwa3B9ex4LGv-zMKsIglkNIyWsXeG9RXsP05rYmLpgkdw6z-V
strict
set-cookie: AEC=AdJVEat6oUeQPLx9DX42axoYHudol4g3Le-oJWYyc2RgRQxQin1LwRLGulM; ex
set-cookie: NID=534=rG5XbQi31UT4xMLHf51nBai0EqibT5U4uueAilsvAURGnk7P3KtUd9rJSt9
BilgSC5DiygPrfLVjvQ; expires=Tue, 16-Feb-2027 21:06:20 GMT; path=/; domain=.google
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
```

Prueba 3: el usuario intenta acceder a un sitio bloqueado

~ curl -x http://192.168.122.1:3128 -I https://www.facebook.com

(Tiene que devolver HTTP/1.1 403 Forbidden. Squid leyó el dominio y le cortó el acceso).

```
root@UTN-Lucka:~# curl -x http://172.20.0.254:3128 -I https://www.facebook.com
HTTP/1.1 403 Forbidden
Server: squid/6.13
Mime-Version: 1.0
Date: Mon, 17 Aug 2026 21:06:29 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 3069
X-Squid-Error: ERR_ACCESS_DENIED 0
Vary: Accept-Language
Content-Language: en
Cache-Status: UTN-Lucka
Via: 1.1 UTN-Lucka (squid/6.13)
Connection: keep-alive
curl: (56) CONNECT tunnel failed, response 403
```

Conclusión

En este laboratorio, implementamos una verdadera arquitectura de borde (*Edge*). Comprobamos por qué el modelo clásico de proxy transparente y reglas iptables lineales quedó atrás, migrando hacia nftables y un proxy explícito con inspección SNI para el tráfico HTTPS.

La red interna ahora carece de acceso directo a Internet, opera bajo un modelo de Confianza Cero donde las peticiones de capa 7 son revisadas, autorizadas o denegadas, y registradas para su análisis.